

UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Complexity, Stacks, Queues, Lists

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

Question 1.1.1: What is an abstract data type?

In computer science, an **abstract data type** is a mathematical **specification** of a set of data and the **set of operations** that can be performed on it. This data type is termed 'abstract' because it serves as a specification that a data structure must then implement.

What is the closest implementation we have in Java for ADT ?

- A Class ?
- An Object ?
- An Interface ?
- A Structure ?
- A primitive Type ?

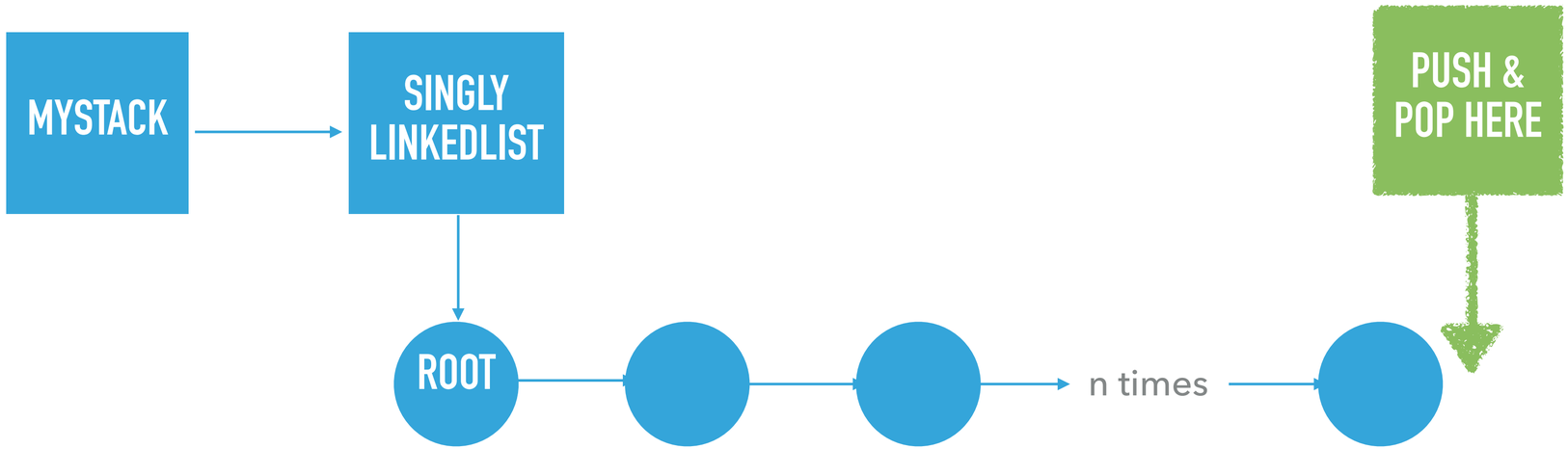
1.1.2 Stack with a simply linked list

Implem of a stack with a simply linked list, where operations are done at the end of the list?

Complexity of a push/pop ? ?

$$\mathcal{O}(1) \quad \sim 1 \quad \Omega(1) \quad \Theta(1)$$

$$\mathcal{O}(n) \quad \sim n \quad \Omega(n) \quad \Theta(n)$$



Question 1.1.3: What are possible implementation for a Stack ?

Java choice ?

java.util.Stack is implemented with an array (extends Vector), why ?

Is this so inefficient 🙀?

	Linked List	ARray
Push	O(1)	O(1) if no resize O(n) if resize
Pop	O(1)	O(1) if no resize O(n) if resize
n push	O(n)	$O(n^2)$ 🤔 ??
n pop	O(n)	$O(n^2)$ 🤔 ??

Stack with an array: a more fine grained analysis

	Liste chainée	Tableau
Push	O(1)	O(1) si pas de redim O(n) si redim
Pop	O(1)	O(1) si pas de redim O(n) si redim
n push	O(n)	O(n²) 🤔 ??
n pop	O(n)	O(n²) 🤔 ??

Doubling tte size whenever we reach the size of the array

On n pushes, we do a resize at these length:

- 1
- 2
- 4
- 8
- 16
- ...
- n

Thus for the n push operations, we did $\log_2(n)$ resize. Each resize cost at most $\mathcal{O}(n)$ thus overal, the n pushes are in $\mathcal{O}(n \log(n))$.

Ca we be more accurate ?

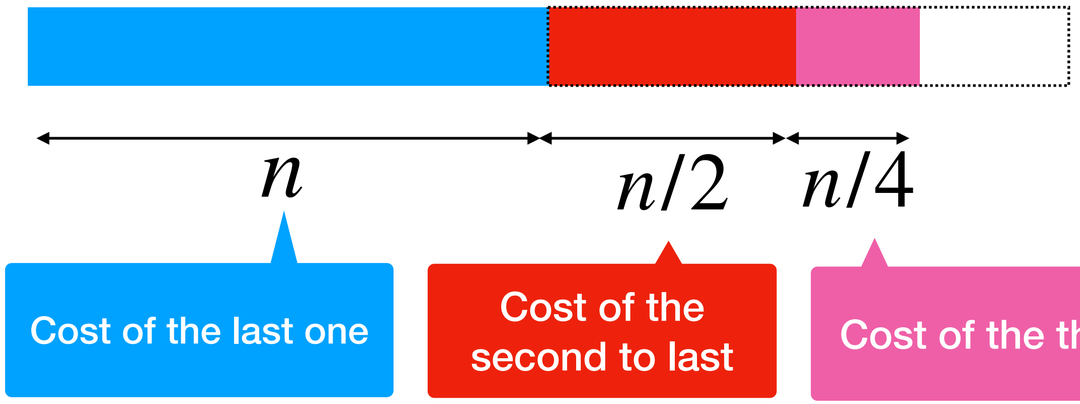
Stack avec tableau: on raffine un peu l'analyse

	Liste chainée	Tableau
Push	O(1)	O(1) if no resize O(n) if resize
Pop	O(1)	O(1) if no resize O(n) if resize
n push	O(n)	O(n log(n)) 🤔 ??
n pop	O(n)	O(n log(n)) 🤔 ??

Doubling tte size whenever we reach the size of the array

On n pushes, we do a resize at these length:

- 1
- 2
- 4
- 8
- 16
- …
- n



Conclusion

java.util.Stack a de très bonnes complexités amorties pour n operations

	Liste chainée	Tableau
Push	O(1)	O(1) Sauf redim. O(n)
Pop	O(1)	O(1) Sauf redim. O(n)
n push	O(n)	O(n)
n pop	O(n)	O(n)

On double la taille du tableau quand on atteint la limite.

Sur n push, on fait donc un resize à ces positions:

- 1
- 2
- 4
- 8
- 16
- ...
- n

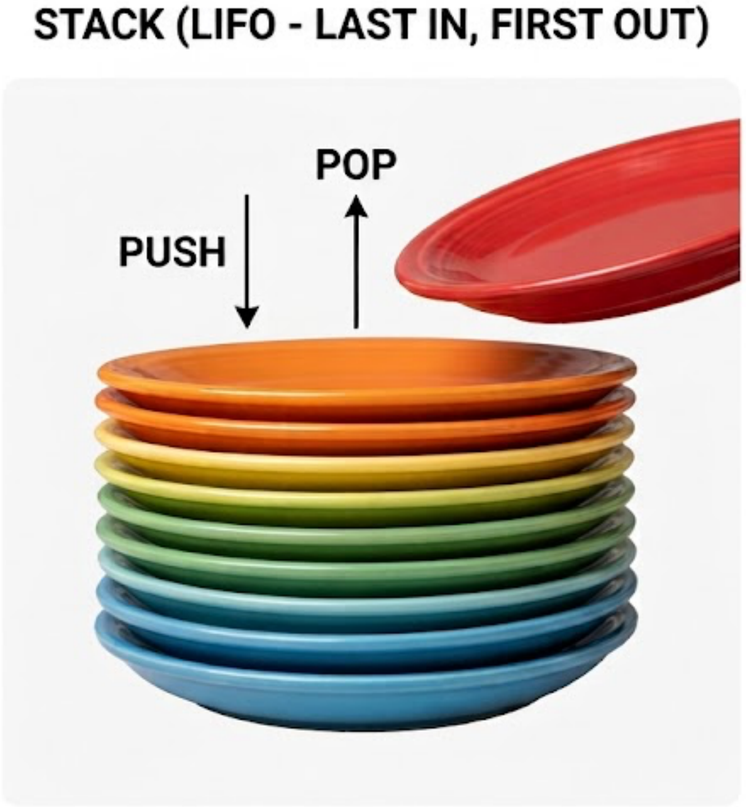
Combien de redim.?

$$\sum_{i=0}^{\log_2 n} 2^i = 2n - 1 \in \mathcal{O}(n)$$

Amortized Time Complexity

Question 1.1.4: Implem of Stack with two Queues

- Reminder



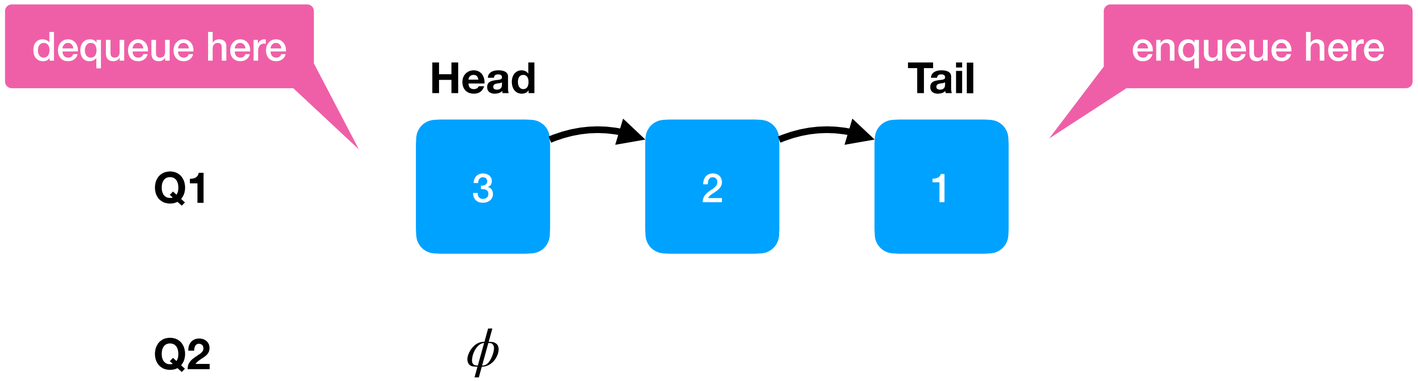


push(e)
E pop()

enqueue(e)
E dequeue/remove()

Question 1.1.4: Idea = use two internal queues

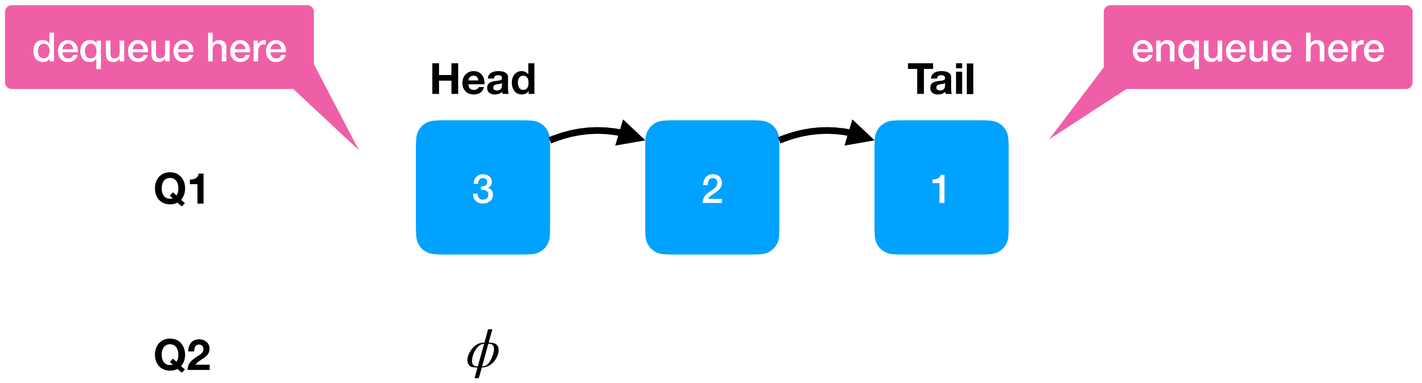
- Let Q1 and Q2 be the two internal queues
- We maintain the invariant that: *Q1 contains the elements of the stack from top (head) to the bottom (tail), Q2 is empty*
- Assume we did push(1) then push(2) then push(3). According to invariant the state is



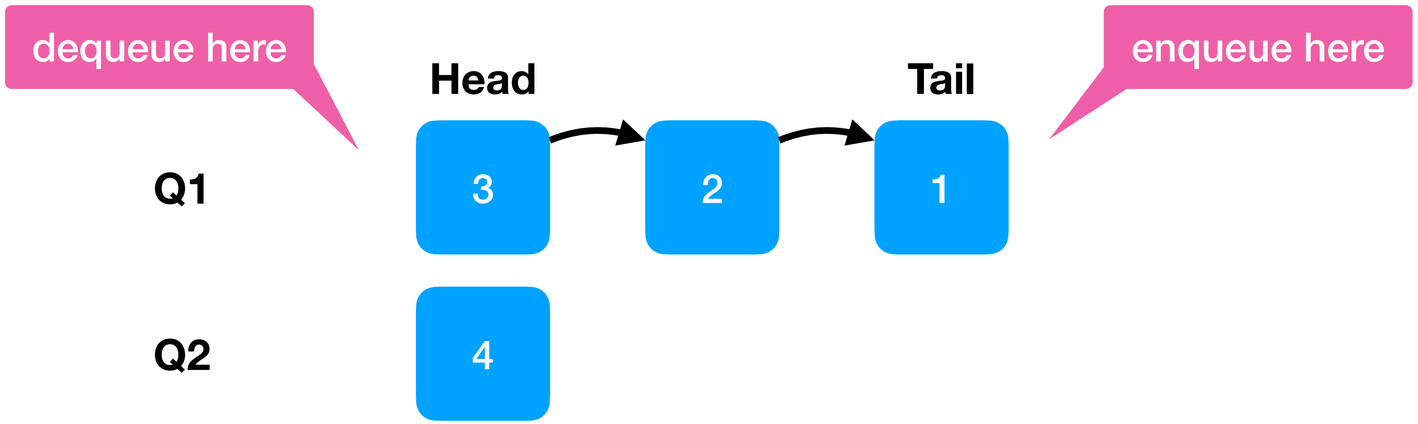
- If I do one more push(4) … then I need to:

1. Add 4 (enqueue) in Q2,
2. Empty Q1 in Q2
3. Swap the references between Q1 and Q2

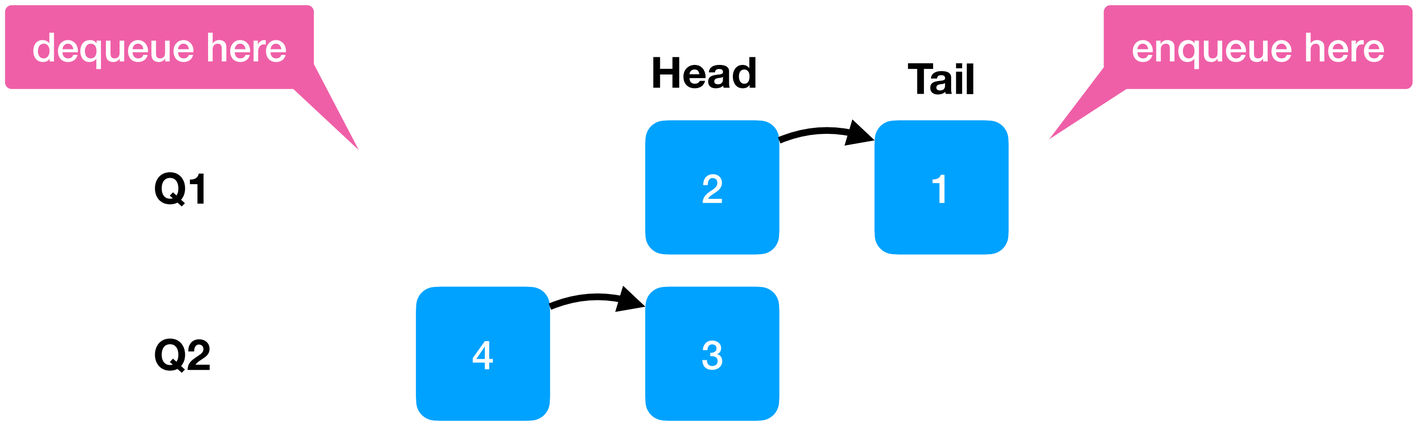
push(4)



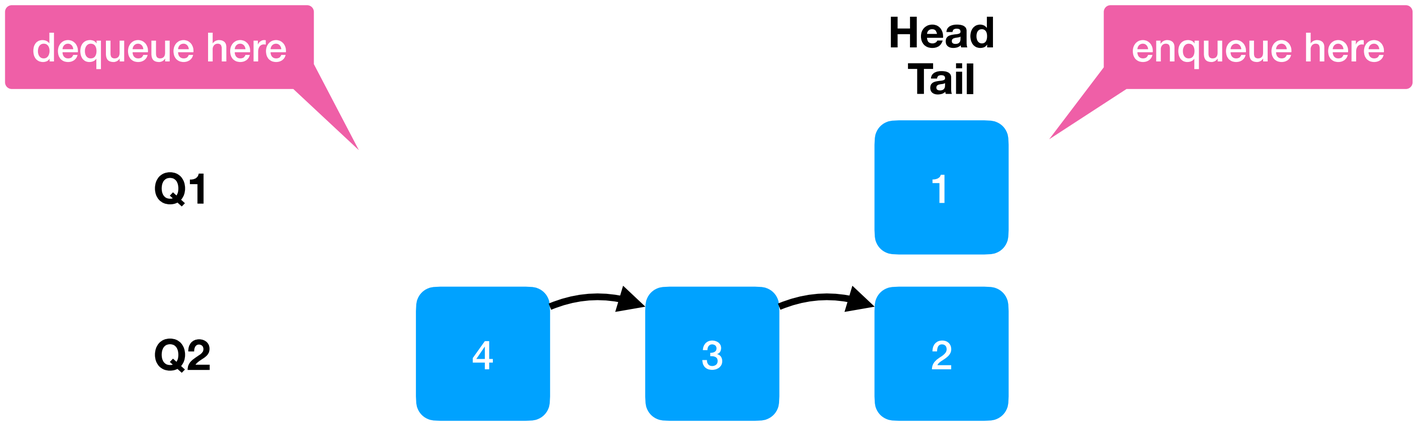
push(4): Q2.enqueue(4)



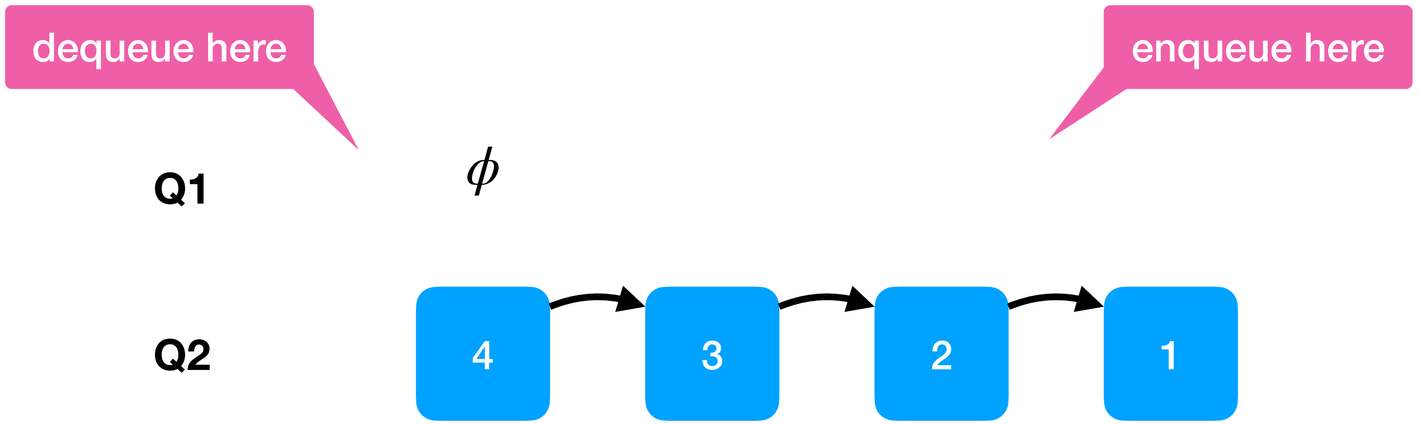
push(4): Q2.enqueue(Q1.dequeue())



push(4): Q2.enqueue(Q1.dequeue())

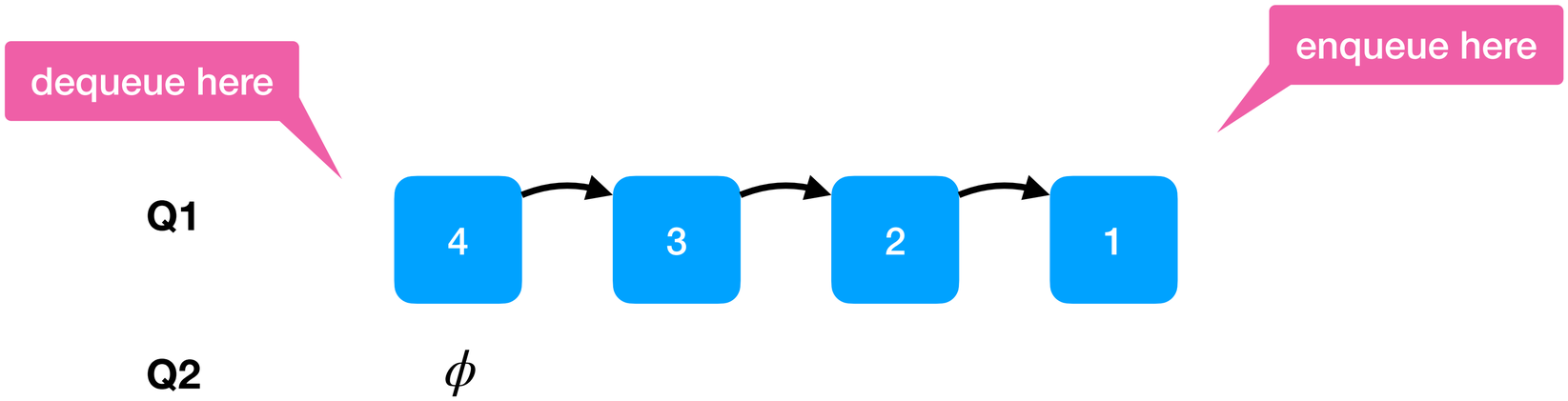


push(4): Q2.enqueue(Q1.dequeue())



push(4): exchange Q1 and Q2

- Invariant is established again!



Question 1.1.4: Stack with two Queues

```
Queue<E> queue1;
Queue<E> queue2; // always empty

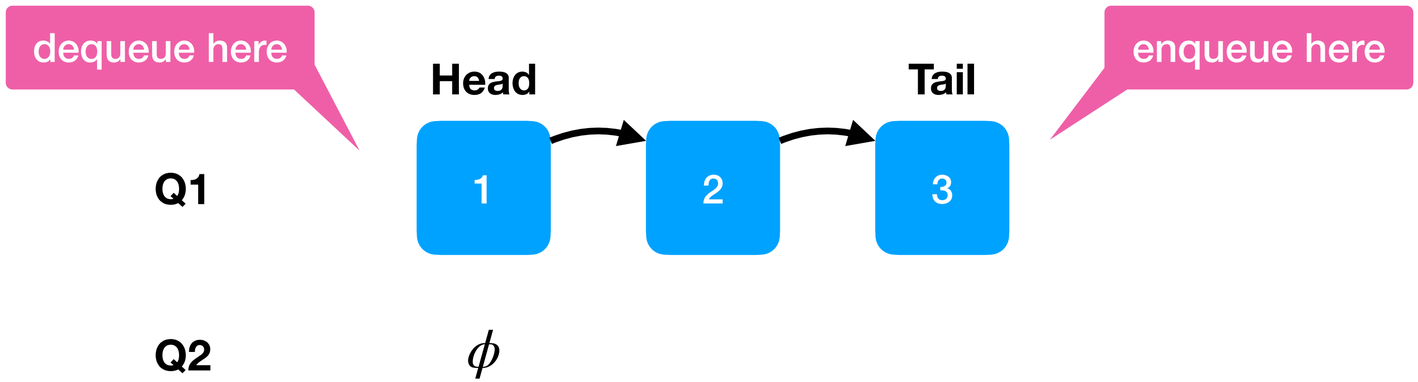
public E pop() throws EmptyStackException {
    if (empty()) throw new EmptyStackException();
    return queue1.remove();
}

public void push(E item) {
    queue2.add(item);
    while (!queue1.isEmpty()) {
        queue2.add(queue1.remove()); // remove = dequeue
    }
    Queue<E> buffer = queue1;
    queue1 = queue2;
    queue2 = buffer;
}
```

- Complexity of push / pop ?
  - $\Theta(n)$ for a push, $\mathcal{O}(1)$ for a pop

Other solution

- Invariant: *Q1 contains the elements of the stack from bottom (head) to the top (tail), Q2 is empty*
  - Situation after push(1), push(2), push(3)



- Complexity push/pop ?
- $\mathcal{O}(1)$ *for a push,* $\Theta(n)$ *for a pop*

Question 1.1.5: Are the three loops having the same time complexity ?

```java
List<Integer> list = new LinkedList<>();

// assume I insert n elements in the list here

for (Integer val: list) {
    System.out.println(val);
}

Iterator<Integer> itr = list.iterator();
while (itr.hasNext()) {
    Integer val = itr.next();
    System.out.println(val);
}

for (int i = 0; i < list.size(); i++) {
    Integer val = list.get(i);
    System.out.println(val);
}
```

Question: Change one word in the source code to get the time time complexity ?

Let’s look into the byte code of

```java
package fundamentals;

import java.util.LinkedList;
import java.util.List;

public class Test {
    public static void main(String[] args) {
        List<Integer> list = new LinkedList<>();
        list.add(1);
        list.add(2);
        for (Integer i : list) {
            System.out.println(i);
        }
    }
}
```

Decompile with javap -v

```
public static void main(java.lang.String[]);
  descriptor: ([Ljava/lang/String;)V
  flags: (0x0009) ACC_PUBLIC, ACC_STATIC
  Code:
    stack=2, locals=4, args_size=1
       0: new           #7                  // class java/util/LinkedList
       3: dup
       4: invokespecial #9                  // Method java/util/LinkedList."<init>":()V
       7: astore_1
       8: aload_1
       9: iconst_1
      10: invokestatic  #10                 // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
      13: invokeinterface #16,  2           // InterfaceMethod java/util/List.add:(Ljava/lang/Object;)Z
      18: pop
      19: aload_1
      20: iconst_2
      21: invokestatic  #10                 // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
      24: invokeinterface #16,  2           // InterfaceMethod java/util/List.add:(Ljava/lang/Object;)Z
      29: pop
      30: aload_1
      31: invokeinterface #22,  1           // InterfaceMethod java/util/List.iterator:()Ljava/util/Iterator;
      36: astore_2
      37: aload_2
      38: invokeinterface #26,  1           // InterfaceMethod java/util/Iterator.hasNext:()Z
      43: ifeq          66
      46: aload_2
      47: invokeinterface #32,  1           // InterfaceMethod java/util/Iterator.next:()Ljava/lang/Object;
      52: checkcast     #11                 // class java/lang/Integer
      55: astore_3
      56: getstatic     #36                 // Field java/lang/System.out:Ljava/io/PrintStream;
      59: aload_3
      60: invokevirtual #42                 // Method java/io/PrintStream.println:(Ljava/lang/Object;)V
      63: goto          37
      66: return
```

Let’s look into the implementation of LinkedList

```
for (int i = 0; i < list.size(); i++) {
    Integer val = list.get(i);
    System.out.println(val);
}
```

```
public E get(int index) {
    checkElementIndex(index);
    return node(index).item;
}
```

Complexity ?

```
public class LinkedList<E>
        extends AbstractSequentialList<E>
        implements List<E>, Deque<E>, Cloneable, java.io.Serializable
{
    transient int size = 0;

    Pointer to first node.
    transient Node<E> first;

    Pointer to last node.
    transient Node<E> last;
```

```
Node<E> node(int index) {
    // assert isElementIndex(index);

    if (index < (size >> 1)) {
        Node<E> x = first;
        for (int i = 0; i < index; i++)
            x = x.next;
        return x;
    } else {
        Node<E> x = last;
        for (int i = size - 1; i > index; i--)
            x = x.prev;
        return x;
    }
}
```

Solution

```
List<Integer> list = new ArrayList<>();

// assume I insert n elements in the list here

for (Integer val: list) {
    System.out.println(val);
}

Iterator<Integer> itr = list.iterator();
while (itr.hasNext()) {
    Integer val = itr.next();
    System.out.println(val);
}

for (int i = 0; i < list.size(); i++) {
    Integer val = list.get(i);
    System.out.println(val);
}
```

Question 1.1.6: Time complexity reminder

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

« Belongs »
O() is a set of functions

This is a function that counts the number of operations in terms of n.

There exists a k such that k·g(n) is greater than f(n) for sufficiently large n.

Question 1.1.6: Time Complexity reminder

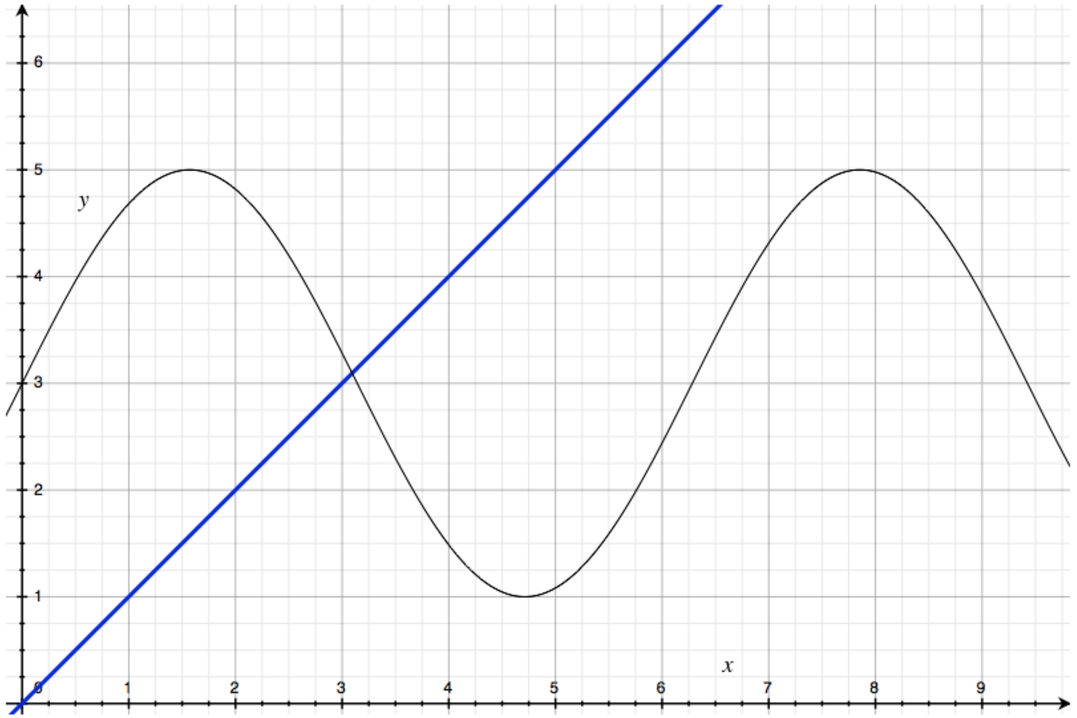
The book uses the ~ (tilde) notation.

In other courses, the notations O, Ω, and Θ are used.

What is the difference between these different complexity classes?

Question 1.1.6: Time Complexity reminder

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$



$$f(x) = 2\sin(x) + 2$$

$$g(x) = x$$

$$2\sin(x) + 2 \in \mathcal{O}(x)$$

Question 1.1.6: Time Complexity reminder

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Omega}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad \mathbf{k} \cdot \mathbf{f(n)} \geq \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Theta}(g(n)) \iff \exists k_0, k_1 \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad \mathbf{k_0} \cdot \mathbf{g(n)} \leq \mathbf{f(n)} \leq \mathbf{k_1} \cdot \mathbf{g(n)} \quad \forall n \geq n_0$$

Question 1.1.6: Time Complexity reminder

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Omega}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad \mathbf{k} \cdot \mathbf{f(n)} \geq \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Theta}(g(n)) \iff \exists k_0, k_1 \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \textbf{t.q.} \quad \mathbf{k_0} \cdot \mathbf{g(n)} \leq \mathbf{f(n)} \leq \mathbf{k_1} \cdot \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \sim g(n) \iff \lim_{n \to \infty} \frac{f(n)}{g(n)} = 1$$

Question 1.1.7: Doubling RATIO TEST

- Why is it useful ?

- If the time complexity of an algorithme can be expressed as $f(n) \sim an^b \log n$, the the doubling ration test can help us to discover b simply by measuring the execution time for some input sizes n, doubling the size each time (We don't event need to look into the source code, the test is completely black box!).

- We can show that the ration converge to 2^b (at the limit):

$$\text{If } f(n) \sim an^b \log n \quad \text{then} \quad \frac{f(2n)}{f(n)} \sim 2^b$$

Question 1.1.7: Doubling RATIO TEST

n	1000	2000	4000	8000	16000	32000	64000
T(n)	0	0	0.1	0.3	1.3	5.1	20.5
Ratio précédent		~1	~1	~3	4.3	3.92	4.01

$\sim 2^b$ seems to converge to 4. Thus the time complexity seems to be $O(n^2)$ if $f(n) \sim an^2[\log n]$ (in practice we can't be sure if there is a $log(n)$ or not, it won't be very much visible on our measures anyway



UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Complexity, Stacks, Queues, Lists

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be